# CPSC-406 Report

Nathan Carnnahan
Chapman University

March 1, 2026

**Abstract**

## Contents

**1 Introduction** **2**

**2 Week by Week** **2**

2.1 Week 1 . . . . . 2

2.2 Week 2 . . . . . 3

2.3 Week 3 . . . . . 7

**3 Synthesis** **10**

**4 Evidence of Participation** **10**

**5 Conclusion** **10**

# 1 Introduction

# 2 Week by Week

## 2.1 Week 1

### HW1 – DFA Exercises

**Exercise 1** We are given two DFAs $A_1$ and $A_2$.

**Accepted Words Table**

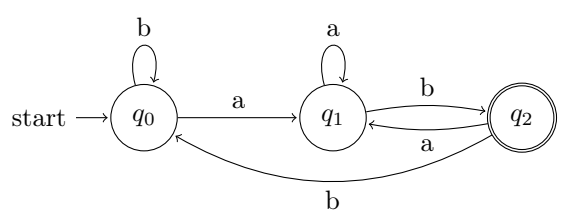
| $w$ | Accepted by $A_1$? | Accepted by $A_2$? |
|---|---|---|
| $aaa$ | No | Yes |
| $aab$ | Yes | No |
| $aba$ | No | No |
| $abb$ | No | No |
| $baa$ | No | Yes |
| $bab$ | No | No |
| $bba$ | No | No |
| $bbb$ | No | No |

**Language Descriptions**

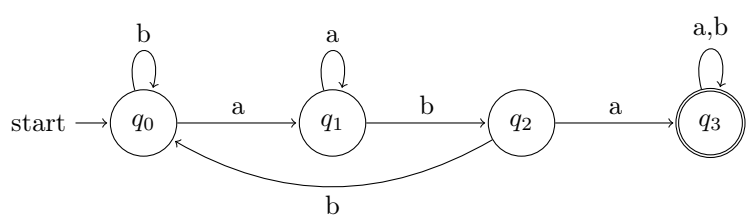
- $L(A_1)$: all strings over $\{a, b\}$ that start with $a$ and end with an odd number of $b$'s.
- $L(A_2)$: all strings over $\{a, b\}$ that end with at least two consecutive $a$'s.

**Exercise 2 – Designing DFAs**

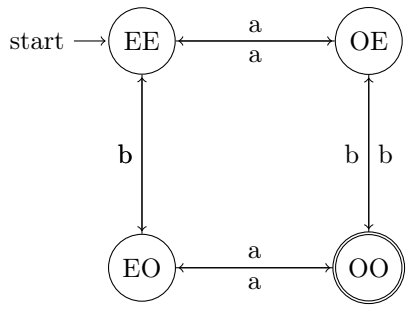
**1. Words that end with $ab$**




**2. Words that contain $aba$**

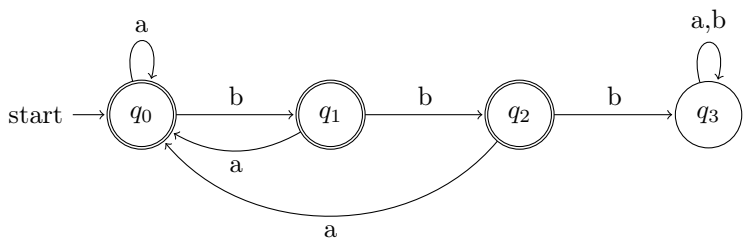



**3. Odd number of $a$'s and odd number of $b$'s** States represent parity: ($a$-parity, $b$-parity).

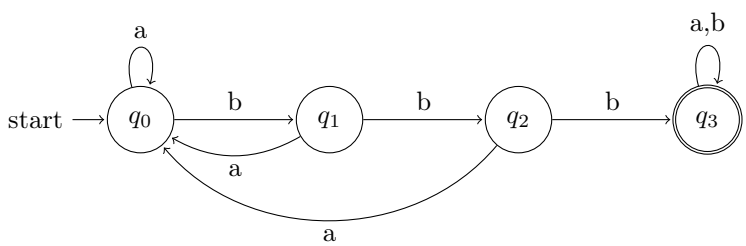



**4. Even number of $a$'s and odd number of $b$'s** Same automaton as above, but accepting state is EO.

**5. Any three consecutive characters contain at least one $a$** Equivalent to forbidding substring $bbb$.




**6. Words that contain $bbb$**




**Observation**

- Problems 3 and 4 use the same parity structure; only the accepting state changes.
- Problems 5 and 6 use the same "count consecutive $b$'s" structure; one treats reaching three $b$'s as rejection, the other as acceptance.
- Problems 1 and 2 track progress toward matching a pattern.

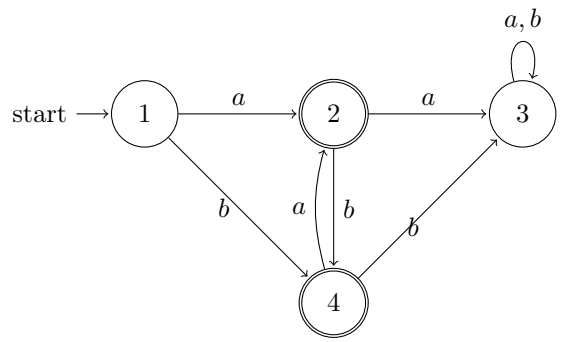
## 2.2 Week 2

**HW2 – Operations on automata**

**Exercise 1 (Product automata).** Throughout, $\Sigma = \{a, b\}$.

**1. Description of $L(A^{(1)})$ and $L(A^{(2)})$.**

<u>$A^{(1)}$:</u>




The accepting states are $\{2, 4\}$ and state 3 is a non-accepting sink (it loops on both $a$ and $b$). From state 2, reading $a$ goes to the sink; from state 4, reading $b$ goes to the sink. Thus, once the first symbol is read, the letters must alternate in order to avoid the sink.

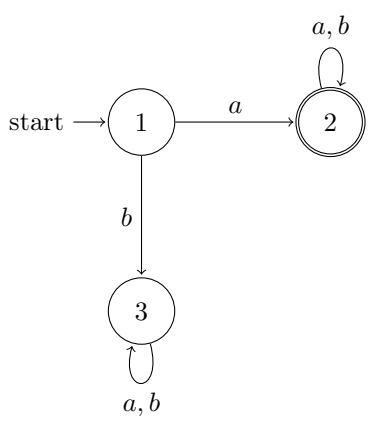
Therefore,

$$L(A^{(1)}) = \{\, w \in \{a, b\}^* : |w| \geq 1 \text{ and } w \text{ contains no substring } aa \text{ or } bb \,\}.$$

Equivalently,

$$L(A^{(1)}) = a(ba)^*(\varepsilon \mid b) \;\cup\; b(ab)^*(\varepsilon \mid a).$$

<u>$A^{(2)}$:</u>




The only accepting state is 2. From state 1, reading $b$ leads to a sink. Reading $a$ moves $1 \to 2$, and from 2 any symbol returns to 1.

Thus a word must:

- start with $a$,
- have odd length,
- have $a$ in every odd position.

Hence,

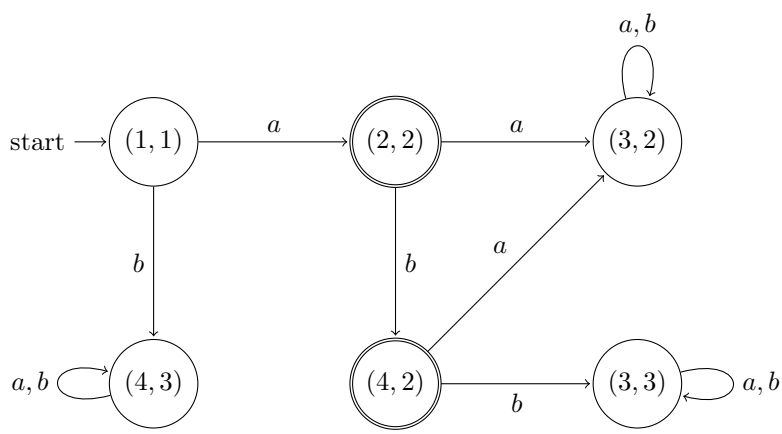
$$L(A^{(2)}) = a\big((a \mid b)a\big)^*.$$

**2. Intersection automaton $A = A^{(1)} \times A^{(2)}$.**

- States: $Q = Q_1 \times Q_2$

- Start state: $(1,1)$
- Accepting states:
$$F = F_1 \times F_2 = \{2,4\} \times \{2\}$$
- Transition function:
$$\delta((p,q),x) = (\delta_1(p,x), \delta_2(q,x))$$

The reachable accepting state is $(2,2)$.




**3. Proof that $L(A) = L(A^{(1)}) \cap L(A^{(2)})$.**

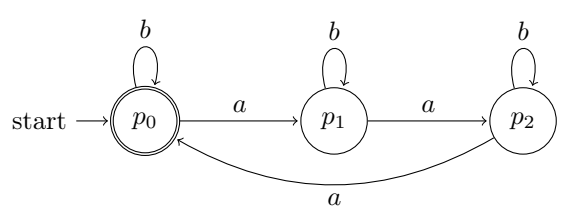
For any word $w$, by induction on prefixes, after reading $w$ the product automaton is in state
$$(\delta_1(1,w), \delta_2(1,w)).$$
Thus $A$ accepts $w$ iff both component automata accept $w$. Therefore,
$$L(A) = L(A^{(1)}) \cap L(A^{(2)}).$$

**4. Construction of union automaton $A'$.**

Keep the same product states and transitions, but define the accepting set as
$$F' = (F_1 \times Q_2) \ \cup \ (Q_1 \times F_2).$$
Then $A'$ accepts whenever at least one component accepts, so
$$L(A') = L(A^{(1)}) \cup L(A^{(2)}).$$

**Exercise 2 (More automata). 1. Description of $L(B^{(1)})$ and $L(B^{(2)})$.**

<u>$B^{(1)}$:</u>



State $p_0$ is accepting. Reading $a$ cycles

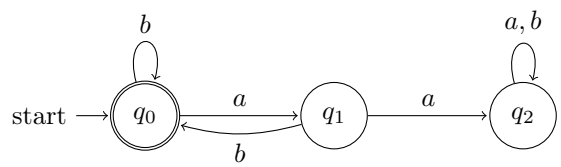
$$p_0 \to p_1 \to p_2 \to p_0,$$

while $b$ loops at each state.

Thus $B^{(1)}$ accepts exactly when the number of $a$'s is divisible by 3:

$$L(B^{(1)}) = \{\, w \in \{a,b\}^* : \#_a(w) \equiv 0 \pmod 3 \,\}.$$

<u>$B^{(2)}$:</u>




State $q_0$ is accepting. The automaton goes to a sink $q_2$ if the substring $aa$ occurs. Also, $q_1$ is non-accepting, so a word ending in $a$ is rejected.

Hence

$$L(B^{(2)}) = \{\, w \in \{a,b\}^* : w \text{ has no substring } aa \text{ and does not end with } a \,\}.$$

Equivalently,

$$L(B^{(2)}) = (b \mid ab)^*.$$

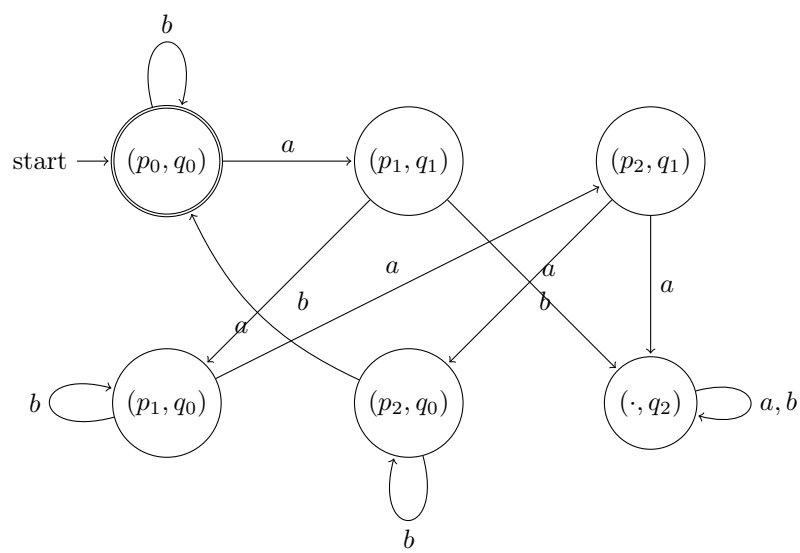
**2. Intersection automaton $B = B^{(1)} \times B^{(2)}$.**

- Start state: $(p_0, q_0)$
- Accepting states:

$$F = \{(p_0, q_0)\}$$

- Transition function:

$$\delta((p_i, q_j), x) = (\delta_1(p_i, x), \delta_2(q_j, x))$$



**3. Proof that $L(B) = L(B^{(1)}) \cap L(B^{(2)})$.**

After reading any word $w$, the product automaton is in state

$$(\delta_1(p_0, w), \delta_2(q_0, w)).$$

Thus $B$ accepts $w$ iff both component automata accept $w$. Therefore,

$$L(B) = L(B^{(1)}) \cap L(B^{(2)}).$$

**4. Construction of union automaton $B'$.**

Using De Morgan's law:

$$L(B^{(1)}) \cup L(B^{(2)}) = \overline{\overline{L(B^{(1)})} \cap \overline{L(B^{(2)})}}.$$

1. Complement $B^{(1)}$ and $B^{(2)}$ by swapping accepting and non-accepting states.
2. Construct their product automaton for intersection.
3. Complement the resulting automaton.

This yields $B'$ such that

$$L(B') = L(B^{(1)}) \cup L(B^{(2)}).$$

**Exercise 2.2.7** Let $A$ be a DFA and let $q$ be a state such that $\delta(q, a) = q$ for all input symbols $a \in \Sigma$. We prove by induction on $|w|$ that for all strings $w \in \Sigma^*$, $\hat{\delta}(q, w) = q$.

*Proof.* **Base case:** $|w| = 0$, so $w = \varepsilon$. By definition of $\hat{\delta}$, $\hat{\delta}(q, \varepsilon) = q$.

**Inductive step:** Assume that for some $n \geq 0$, for every string $x \in \Sigma^*$ with $|x| = n$ we have $\hat{\delta}(q, x) = q$. Let $w \in \Sigma^*$ be any string with $|w| = n + 1$. Then $w$ can be written as $w = xa$ where $|x| = n$ and $a \in \Sigma$ is the last symbol. Using the recursive definition of $\hat{\delta}$,

$$\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a).$$

By the inductive hypothesis, $\hat{\delta}(q, x) = q$, so

$$\hat{\delta}(q, xa) = \delta(q, a) = q,$$

where the last equality uses the assumption that $\delta(q, a) = q$ for all $a \in \Sigma$. Thus $\hat{\delta}(q, w) = q$ for all $w$ of length $n + 1$.

By induction, $\hat{\delta}(q, w) = q$ for all $w \in \Sigma^*$. □

**Interesting Question for Discord (from §2.2.4)** Can the same automaton be setup to accept multiple languages? If so, can we use the same automaton hardware to create a multifunctional dfa?

## 2.3 Week 3

### Assignment 3 – NFAs and Determinization

### Question 1

For the alphabet $\Sigma = \{a, b, c\}$ consider the NFA $A$ shown in the prompt.

**1. Regular expression for $L(A)$.** From the start state 0, we may read any number of $a$'s and $b$'s while staying in state 0. At some point we must read an $a$ to move to state 1, and then read exactly one more symbol ($b$ or $c$) to reach an accepting state. Therefore the language is

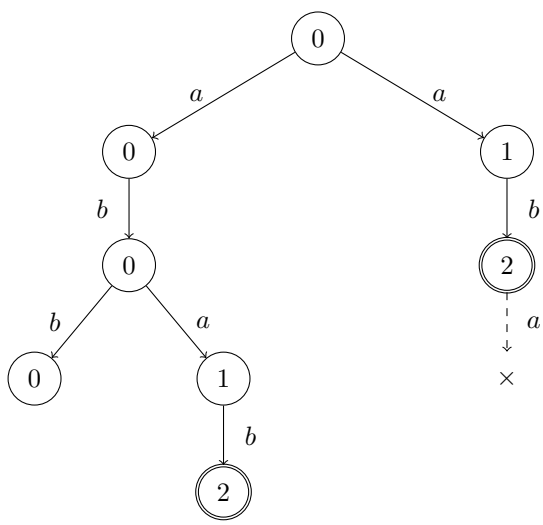
$$L(A) = (a \mid b)^* \, a \, (b \mid c).$$

**2. NFA specification $(Q, \Sigma, \delta, q_0, F)$.**

- $Q = \{0, 1, 2, 3\}$
- $\Sigma = \{a, b, c\}$
- $q_0 = 0$
- $F = \{2, 3\}$
- Transition function $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ given by:

| $q$ | $\delta(q, a)$ | $\delta(q, b)$ | $\delta(q, c)$ |
|---|---|---|---|
| 0 | $\{0, 1\}$ | $\{0\}$ | $\emptyset$ |
| 1 | $\emptyset$ | $\{2\}$ | $\{3\}$ |
| 2 | $\emptyset$ | $\emptyset$ | $\emptyset$ |
| 3 | $\emptyset$ | $\emptyset$ | $\emptyset$ |

**3. All paths for $v = abab$.** Because $\delta(0, a) = \{0, 1\}$, the first symbol $a$ branches into two threads. The diagram below shows all paths; the accepting path is marked with ✓ and the stuck path is marked with $\times$.




The rightmost branch gets stuck after reading $ab$ because state 2 has no outgoing transitions. The only accepting computation is the leftmost path:

$$0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 1 \xrightarrow{b} 2 \checkmark$$

**4. Determinization $A^D$ (powerset construction).** The DFA states are subsets of $Q$. The reachable subsets are

$$\{0\},\ \{0, 1\},\ \{0, 2\},\ \{3\},\ \emptyset.$$

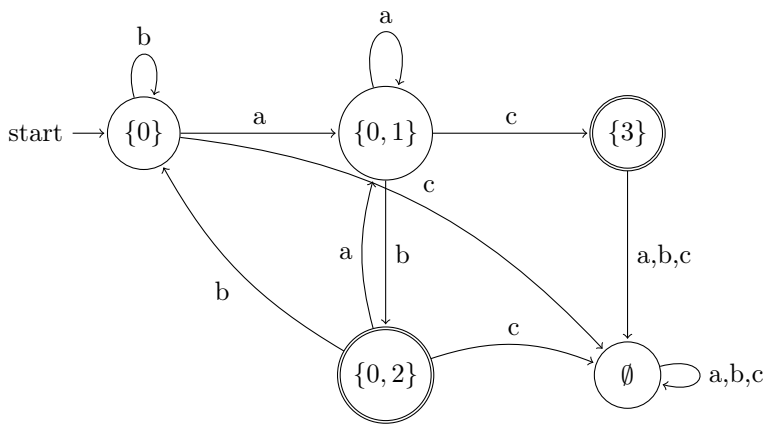
The start state is $\{0\}$ and the accepting DFA states are those subsets containing 2 or 3, i.e.

$$F_D = \{\{0, 2\}, \{3\}\}.$$

**Transition table for $A^D$.**

| $S$ | $\delta_D(S,a)$ | $\delta_D(S,b)$ | $\delta_D(S,c)$ |
|---|---|---|---|
| $\{0\}$ | $\{0,1\}$ | $\{0\}$ | $\emptyset$ |
| $\{0,1\}$ | $\{0,1\}$ | $\{0,2\}$ | $\{3\}$ |
| $\{0,2\}$ | $\{0,1\}$ | $\{0\}$ | $\emptyset$ |
| $\{3\}$ | $\emptyset$ | $\emptyset$ | $\emptyset$ |
| $\emptyset$ | $\emptyset$ | $\emptyset$ | $\emptyset$ |

**Graphical depiction of $A^D$.**




**Question 2**

For the alphabet $\Sigma = \{0,1\}$ consider the NFA $A$ shown in the prompt.

**1. Determinization $A^D$ (powerset construction).** The NFA transitions are:

$$\delta(q_0,0) = \{q_0\},\ \delta(q_0,1) = \{q_0,q_1\},\ \delta(q_1,1) = \{q_2\},\ \delta(q_2,0) = \{q_1,q_3\},\ \delta(q_3,0) = \delta(q_3,1) = \{q_3\}$$

and all other transitions are $\emptyset$.

The reachable DFA states (subsets) are:

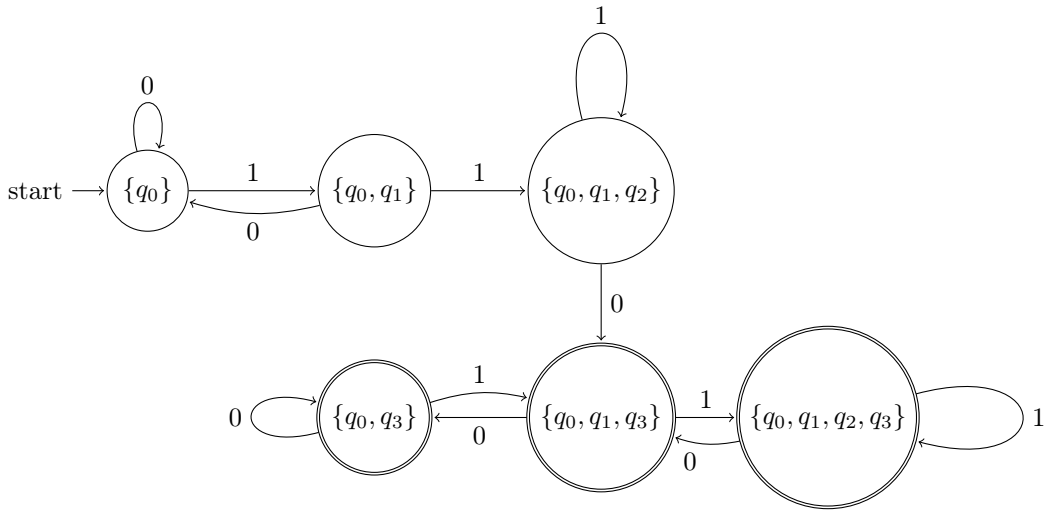
$$\{q_0\},\ \{q_0,q_1\},\ \{q_0,q_1,q_2\},\ \{q_0,q_1,q_3\},\ \{q_0,q_3\},\ \{q_0,q_1,q_2,q_3\}.$$

Start state: $\{q_0\}$. Accepting DFA states are those containing $q_3$.

**Transition table for $A^D$.**

| $S$ | $\delta_D(S,0)$ | $\delta_D(S,1)$ |
|---|---|---|
| $\{q_0\}$ | $\{q_0\}$ | $\{q_0,q_1\}$ |
| $\{q_0,q_1\}$ | $\{q_0\}$ | $\{q_0,q_1,q_2\}$ |
| $\{q_0,q_1,q_2\}$ | $\{q_0,q_1,q_3\}$ | $\{q_0,q_1,q_2\}$ |
| $\{q_0,q_1,q_3\}$ | $\{q_0,q_3\}$ | $\{q_0,q_1,q_2,q_3\}$ |
| $\{q_0,q_3\}$ | $\{q_0,q_3\}$ | $\{q_0,q_1,q_3\}$ |
| $\{q_0,q_1,q_2,q_3\}$ | $\{q_0,q_1,q_3\}$ | $\{q_0,q_1,q_2,q_3\}$ |

**Graphical depiction of $A^D$.**

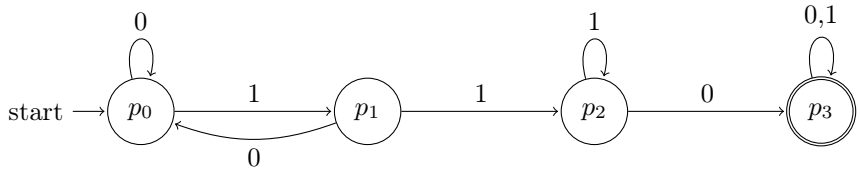



**2. Smaller DFA accepting the same language.** Intuitively, because $q_0$ loops on both 0 and 1, the NFA can "start" its nontrivial behavior at any position. To reach the accepting state $q_3$ we must read the pattern

$$1\,1\,0$$

in consecutive positions (take $q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_3$). Once in $q_3$, the automaton accepts and stays accepting forever. Thus the language is exactly the set of binary strings containing substring 110.

A minimal DFA for "contains 110" has 4 states tracking the longest suffix that is also a prefix of 110:




Here $p_3$ is the accepting sink (we have seen 110).

# 3 Synthesis

# 4 Evidence of Participation

# 5 Conclusion

# References


[BLA] Nathan Carnnahan, CPSC-406 Report, Nathan Carnnahan, 2026.